Heap Data Structures Implementation and Applications

Furqaanahmed Khan

University of the Cumberlands

MSCS 532: Algorithms and Data Structures

Professor Michael Solomon

November 10, 2026

Heap Data Structures Implementation and Applications

A binary heap supports both in-place sorting and efficient selection of the next task. This implementation compares Heapsort with randomized Quicksort and Merge Sort, then applies a max-heap to nonpreemptive priority scheduling. The measured examples illustrate both the cost of heap operations and the fact that priority alone does not guarantee deadlines.

Heapsort Implementation and Analysis

The list represents a complete binary tree: for index i, the children are 2i + 1 and 2i + 2. Bottom-up construction sifts each internal node down until the parent is at least as large as its children. The algorithm then exchanges the maximum at the root with the last heap element, reduces the heap boundary, and sifts the new root down. The suffix grows in ascending order while the remaining prefix stays a max-heap (Cormen et al., 2022).

Building the heap takes O(n): at most about n/2^(h+1) nodes have height h, and the sum of h/2^(h+1) is constant. Each of n − 1 extractions takes O(log n), giving O(n log n) worst-case time and the same upper bound for average and best cases. For distinct keys, standard Heapsort has Θ(n log n) best, average, and worst running time. The implementation stops sifting when the parent is already large enough, however, so an all-equal input takes Θ(n) even during extraction. Thus O(n log n) remains a valid upper bound for every case, but it is not a tight lower bound for every input with duplicates.

Heapsort rearranges the input list and uses O(1) auxiliary space: its iterative sift stores a constant number of indexes and temporary references. It is not stable because exchanges can reverse equal items. Python list storage and the benchmark’s input copies are separate from the algorithm’s auxiliary space.

Empirical Comparison

The benchmark used Python 3.12.12 on macOS arm64. It compared the same seeded permutation, sorted list, and reverse-sorted list at each size. Five runs per case were timed with a high-resolution clock; the median is shown in milliseconds. Copying inputs and checking results against Python's sorted output occurred outside the timed interval. Quicksort used random pivots and three-way partitioning; Merge Sort used a bottom-up auxiliary array. Pivot seeds were reset for reproducibility.

Input	Size	Heap ms	Quick ms	Merge ms
random	500	0.428	0.436	0.383
random	1000	1.007	0.929	0.803
random	2000	2.228	1.951	1.683
random	4000	4.898	4.342	3.630
sorted	500	0.427	0.390	0.294
sorted	1000	1.015	0.876	0.691
sorted	2000	2.278	1.904	1.436
sorted	4000	5.127	4.081	3.050
reverse	500	0.363	0.391	0.289
reverse	1000	0.872	0.852	0.640
reverse	2000	2.023	1.890	1.405
reverse	4000	4.637	3.991	2.990

At 4,000 elements, Heapsort took 4.898 ms on random input, compared with 4.342 ms for Quicksort and 3.630 ms for Merge Sort. Merge Sort was fastest in all recorded cases; these small differences reflect the specific Python implementations and machine, not a universal ranking. All three grew substantially as input size increased, consistent with their $O(n \log n)$ bounds for these cases. Randomized Quicksort has expected $O(n \log n)$ time but can take $O(n^2)$ with unfavorable pivots; Merge Sort takes $\Theta(n \log n)$ time and $O(n)$ auxiliary space. One input per case and five timings do not capture machine noise or variation across independent datasets.

Priority Queue and Scheduling

The queue stores tasks in a list-based max-heap, with a dictionary mapping each unique task ID to its current index. A list compactly represents the complete tree without child pointers, and the index map avoids a linear search for a task whose priority changes. Each immutable task

has an ID, priority, arrival time, deadline, and duration. Higher numbers mean higher priority; equal-priority tasks leave in insertion order. The queue rejects duplicate IDs and signals an empty extraction or unknown ID with an exception.

Insertion appends and sifts upward; extraction swaps the root with the last entry, removes it, and sifts downward. Each takes O(log n) heap work. Changing a priority uses the map to find the task, then sifts up after an increase or down after a decrease, also O(log n) expected time with expected O(1) dictionary lookup. List growth and dictionary operations are amortized/expected; an individual resize can cost O(n), and dictionary worst cases are not constant. Checking emptiness takes O(1). The heap and index map require O(n) space.

The simulation sorts arrivals and runs the highest-priority available task to completion. When idle it advances the clock to the next arrival. Tasks arriving during execution wait; deadlines are measured at completion and do not determine priority. Sorting and the queue operations take O(n log n) expected time in total, with O(n) space.

A runs from 0 to 3; B and D, both priority 5, follow in arrival order, finishing at 5 and 7. C waits five units and finishes at 8, missing its deadline of 7. E arrives at 10 after two idle units and finishes at 12. Mean wait is 1.8 units; one of five tasks misses its deadline. Higher-priority arrivals delay C, and sustained traffic could starve lower-priority tasks. Nonpreemption can delay urgent arrivals. This example does not represent a real workload.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/